# Assignment 5:

# QuickSort Algorithm: Analysis, implementation and Randomization

Shreeya Gadgil

For MSCS 532 – Algorithms and Data Structures

## A) Deterministic Quick Sort Performance Analysis:

### a. Time complexity Analysis:

i. Methodology: Pick one element from the array as a reference point or pivot. Then rearrange the array such that all values smaller than the pivot move to its left and values larger than the pivot move to the right. This locks the pivot in its sorted position. Then apply the same steps separately to the left sub-array and right sub-array. Stop the process when a sub-array has a single element, since a single item is already considered sorted.

ii. **Mathematical analysis:**

- **Best Case:**

  The general recurrence relation for Quicksort is: T(n) = T(k) +T(n-1-k) + Θ(n) where k is the number of elements in the left partition. In the best-case scenario, the pivot divides the array into two equal halves. At every level, the partition function scans the entire length to compare elements against the pivot. This takes O(n) time. Since the array is split 'n' half each time, the number of times you can divide the array before reaching individual elements is log(n) levels. This gives the total time as O(nlog(n)).

  Then the recurrence relation is: T(n) = 2T(n/2) + Θ(n)

  **Master Method:**

  - Here, a =2, b=2 and f(n) = Θ(n) = Θ ($n^1$), so d = 1.
  - Calculate $n^{\log_b a} = n^{\log_2 2}$ = n
  - Since f(n) = Θ(n), this falls into Case 2 of the master theorem.
  - Therefore, T(n) = Θ($n^{\log_b a} \log n$) = Θ(nlogn)

- **Average case:**

  For arbitrary inputs as well, the splits tend to be reasonably balanced. This would still result in the recursion tree height being proportional to log(n). Hence the average case runtime is O(nlog(n).

  We can assume that every possible permutation of the input is equally likely, meaning the pivot is equally likely to end up anywhere from 0 to n-1.

  **Recurrence relation:**

$$T(n) = \Theta(n) + \frac{1}{n}\sum_{k=0}^{n-1}[T(k) + T(n-1-k)]$$

  Since T(k) and T(n-1-k) generate the same terms, we can simplify this as:

$$T(n) = cn + \frac{2}{n}\sum_{k=0}^{n-1}[T(k)]$$

  Next, we derive the time complexity via Algebraic elimination (as referenced in The Art of Computer Programming, Volume 3: Sorting and Searching by Donald E. Knuth.)

  Multiplying this by n, we get

$$nT(n) = cn^2 + 2\sum_{k=0}^{n-1}[T(k)]$$

  For n-1, the equation would look like:

$$(n-1)T(n-1) = c(n-1)^2 + 2\sum_{k=0}^{n-2}[T(k)]$$

Subtracting the second equation from the first,

$$nT(n) - (n-1)T(n-1) = \mathrm{c}(2\mathrm{n}-1) + 2\,(T(n-1))$$

Therefore,

$$nT(n) = \mathrm{c}(2\mathrm{n}-1) + (\mathrm{n}+1)(T(n-1))$$

Dividing both sides by n(n+1) and further approximating (2n-1)/n(n+1) as 2/n, we get

$$\frac{T(n)}{n+1} \approx \frac{T(n-1)}{n} + \frac{2c}{n}$$

Expanding this telescoping series:

$$\frac{T(n)}{n+1} \approx \frac{T(0)}{1} + 2c\sum_{i=10}^{n} 1/i$$

The summation here is the Harmonic series, which scales to ln(n). Therefore,

$$\frac{T(n)}{n+1} \approx 2c(\ln n)$$

$$T(n) \approx 2c(\ln n)(n+1) = \Theta(\mathrm{nlogn})$$

- **Worst Case:** The pivot is always the smallest or largest element in the array(in case of a reverse sorted array) and the array is split into one empty side and one side with n-1 elements. This can lead to the algorithm recursing n times, resulting in work which increases the complexity to $O(n^2)$

  **Recursion Tree:**

  - Level 0 = cn → Level 1 = c(n-1) → Level 2 = c(n-2) → Level n-1 = c(1)
  - Total work would be T(n) = $c\sum_{i=1}^{n} i$ = $c(n + n^2)/2 = \Theta(n^2)$

**b. Space complexity and additional overheads:**

Quicksort modifies data in place without needing a separate array, but it does require space to track active frames. In the best case, the algorithm achieves a space complexity of Θ(logn) because the recursion tree splits evenly, limiting the height of a balanced binary tree to $\log_2 n$. The average case also has a space complexity of Θ(logn) because even if unequal splits occur, the average depth of the recursion tree balances out while the algorithm runs. The worst case, however, degrades to a space complexity of O(n) when the tree splits unevenly. Here, the recursion sequentially calls T(n-1) and T(n-2) which pushes n active frames onto the stack.

Memory overheads for deterministic Quicksort can stem from advanced pivot selection strategies like the “Median-of-three”, designed to avoid breakdown in worst case scenarios. Calculating medians requires extracting multiple elements, comparing and swapping them before the main partitioning loop begins. Furthermore, when deterministic quicksort needs to sort data that results in uneven partitions like almost sorted or repetitive data, the CPU’s branch predictor may fail to accurately predict the outcome of comparisons. This causes the processor to stall and ignore pre-fetched work. If the pivot rule consistently results in poor splits (like in the worst-case scenario), the stack of recursive calls expands linearly rather than logarithmically. It creates a long chain of unfinished tasks on the memory which can cause L1/L2 cache misses, slowing the process down further.

## B) Randomized Quick sort Analysis:

### a. Time Complexity:

- **Best Case:** In this case, the pivot divides the array into exactly two equal parts. Calculated in a similar way as Deterministic Quicksort, resulting in a best case time complexity of Θ(nlogn).
- **Average Case:**

  Let Z = [z1, z2, z3, ..., zn] denote the elements of an array A sorted in ascending order. Let Zij = {zi, zi+1,... , zj} be the set of elements between zi and zj both inclusive.

  We define an indicator random variable Xij as:

  $X_{ij}$ = zi is compared to zj = $\begin{cases} 1, & if\ zi\ is\ compared\ to\ zj \\ 0, & otherwise \end{cases}$

  The total number of comparisons X is the sum of all pairs:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

  **Linearity of Expectation:**

  Taking the expected value of both sides

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}]$$

  Since $\mathbb{E}[X_{ij}] = 1 \,.\, \mathbb{P}(when\ zi\ is\ compared\ to\ zj) + 0 \,.\, \mathbb{P}(when\ zi\ is\ not\ compared\ to\ zj)$, we have

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{P}(when\ zi\ is\ compared\ to\ zj)$$

  **Probability calculation:**

  Two elements zi and zj are compared only if the first element chosen as a pivot from the subset is either one of them. If any element between zi and zj is chosen first, zi and zj will be separated into different sub-arrays and will never be compared. As the total number if elements is j-i+1, when the pivot is chosen uniformly at random, every element in Zij has an equal probability of 1/(j-i+1) of being chosen first. Hence the probability of either zi or zj being chosen as the first pivot is 2/(j-i+1)

  Substituting the probability back into the expectation formula:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$$

  Let k = j – i. As j goes from i+1 to n, k goes from 1 to n-i:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n} 2 \,.\, H_n$$

  $where\ Hn = \sum_{k=1}^{n} \frac{1}{k}$ is the nth Harmonic number, bound by ln(n) +O(1)

  $\mathbb{E}[X] < 2n \,.\, H_n = 2n\big(\ln(n) + O(1)\big)$ = Θ(nlogn)

  This proves that the average case time complexity of Randomized Quicksort is O(nlogn).

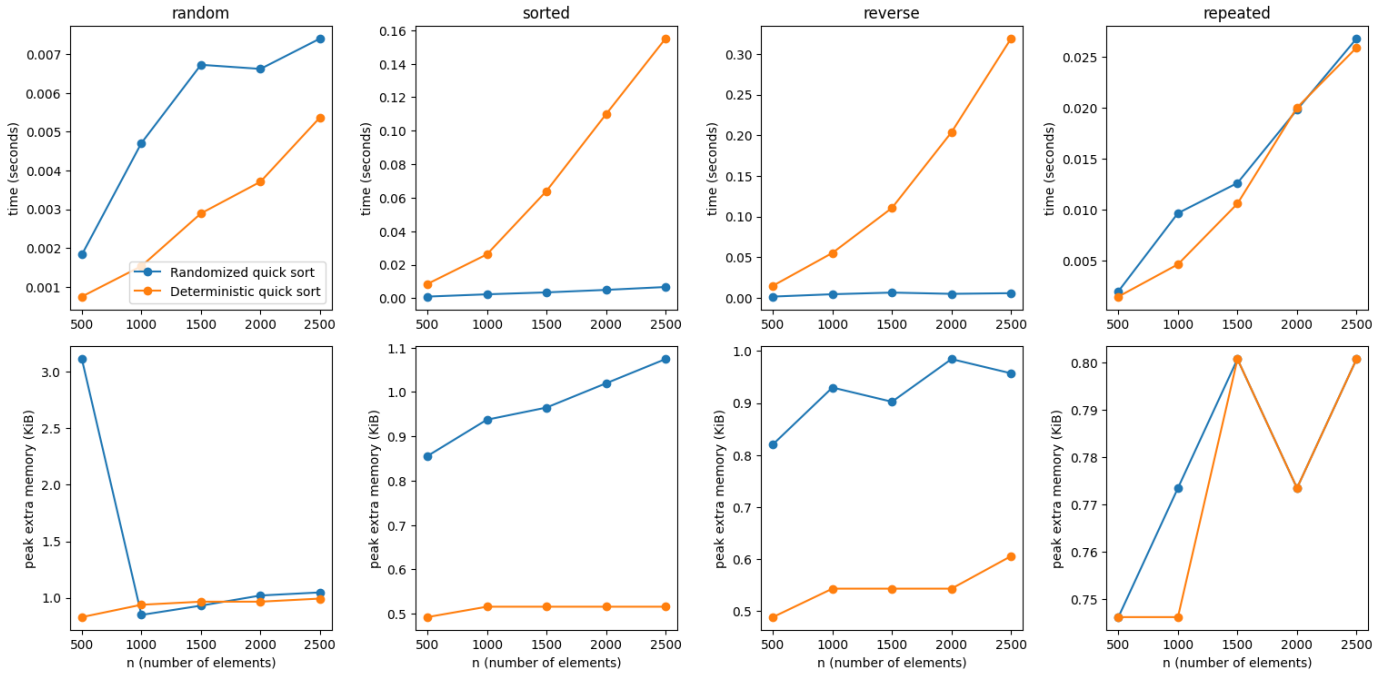
- **Worst Case:** The worst case runtime occurs if the random number generator picks the minimum or maximum element as a pivot at every recursive step. This yields the exact same calculation as deterministic quicksort leading to $\Theta(n^2)$ time complexity.

**b. Effect of Randomization on the performance of Quicksort:**

Theoretically, even though the time complexities of both styles of Quicksort are $\Theta(n^2)$ in the worst case, randomized quicksort is highly unlikely of encountering such a scenario when it randomly chooses a pivot at each step. Deterministic quicksort has a predictable failure mode, which can be misused by attackers in sensitive applications.

Randomization also helps to prevent memory overflows, since the call stack depth stays bounded to O(logn) space with extremely high probability. This is due to the random pivots leading to reasonable balanced partitions most of the time. While randomization guarantees such stability, it introduces minor hidden costs. Using a random number generator adds computing overhead, and jumping to random elements can cause the CPU to miss its fast hardware cache lines, meaning a randomized quicksort can sometimes run slightly slower on clean, completely random data than a tuned deterministic algorithm.

## C) Empirical Comparison:




Both algorithms share the same partition logic and the same recursion structure. The only difference is the pivot rule. Both were benchmarked across five input sizes and four data set types of random, sorted, reverse sorted and repeated elements.

On random input, both algorithms grow with n as expected, but deterministic quicksort is consistently slightly faster than randomized quicksort. This occurs because the first element of a random subarray is itself effectively a random pivot, allowing deterministic quicksort to achieve the same average case behaviour while randomized quicksort requires the extra step of calling random.randint on every partition call. However, on sorted and reverse sorted data, the two algorithms diverge significantly; deterministic quicksort degrades completely to its worst case, while randomized quicksort stays low and flat on both. This divergence proves that a fixed pivot rule can be highly vulnerable to specific inputs, whereas a randomized rule is not. Finally, when handling arrays with repeated elements, rather than one algorithm staying flat and the other exploding, both algorithms grow together. Tracing through the partition function on an array of all-equal elements shows that the test a[j] <= x is True for every single element, regardless of which index was chosen as the pivot. By the time the loop runs, the pivot value is already fixed, and the loop's behaviour depends entirely on that value, not the index itself; hence, randomizing which index gets swapped to the end becomes completely irrelevant when every index holds the exact same value. Ultimately, this traces back to the core assumption made at the start of the average-case proof, which requires distinct elements, meaning that with duplicates, both algorithms degrade to $\Theta(n^2)$.

**Repository : https://github.com/shreeya-up/MSCS532_Assignment5.git**